

On the maximum number of distinct integers representable by substrings of a binary string

Ben Clare

May 31, 2026

Abstract

For a binary string s of length n , let $f(s)$ denote the number of distinct non-negative integers obtained by reading contiguous substrings of s as binary numerals, with leading zeros ignored, and set $a(n) = \max_{|s|=n} f(s)$. This is OEIS sequence A112509.

We prove that $\lim_{n \rightarrow \infty} a(n)/n^2 = \frac{1}{2}$; more precisely, for all $n \geq 36$,

$$\frac{1}{2}n^2 - \frac{13}{4}n^{3/2} \leq a(n) \leq \frac{n(n+1)}{2},$$

so $a(n)/n^2 = \frac{1}{2} + O(n^{-1/2})$. We also construct explicit near-optimal strings of ones-density $1 - O(n^{-1/2})$ and prove that every optimal string has ones-density $1 - O(n^{-1/4})$. The sequence is convex: $a(n+1) - 2a(n) + a(n-1) \geq 0$ for all $n \geq 2$, and its second differences satisfy $\Delta^2 a(n) = O(n^{3/4})$. Finally, we prove that any two consecutive zero-separators of equal length force $\Theta(\min(b_i, b_{i+1}) \cdot \min(b_{i+1}, b_{i+2}))$ duplicate substring values, giving a quantitative obstruction to repeated adjacent separator lengths.

A branch-and-bound algorithm founded on these bounds extends the certified table from Fuller's A* search for $n \leq 80$ to $n \leq 111$. An explicitly scored string at $n = 2 \times 10^9$ gives the certified lower bound $a(n)/n^2 \geq 0.49996$.

Contents

1	Introduction	2
2	Definitions and notation	4
3	Upper bounds	4
4	The elementary lower bound	5
5	The asymptotic limit	5
5.1	Strictly decreasing partitions	5
5.2	The explicit construction	6
5.3	An injective family of substrings	6
5.4	Quantitative lower bound	7
5.5	A sharper quantitative bound	7
6	Density of ones	8
6.1	Near-optimal strings	8
6.2	The zero-position penalty	9
6.3	Optimal strings have few zeros	9
7	Upper bounds from run structure	9

8	The run-length frequency bound	10
9	Convexity of the sequence	11
10	The ones-weighted crossing bound	13
10.1	Suffix-automaton refinement	13
11	Separator structure	14
11.1	Forced collisions from equal consecutive separators	14
12	Computational results	15
12.1	Scoring	15
12.2	Branch-and-bound exact solver	15
12.3	Exact values	17
12.4	Exploratory lower bounds for $112 \leq n \leq 200$	17
12.5	Large-scale certified lower bounds	18
13	Empirical observations and open problems	18
13.1	Second differences	18
13.2	The separator pattern	18
13.3	Open problems	19

1 Introduction

Given a binary string, every contiguous substring can be read as the binary representation of a non-negative integer (with leading zeros suppressed: $01 = 1 = 1$, and $0 = 0$). The question of maximising the number of distinct values simultaneously representable in a single n -bit string was motivated by a competition problem and appears as OEIS A112509 [1].

Definition 1.1. For $s \in \{0, 1\}^n$, let $V(s)$ denote the set of distinct non-negative integers representable by contiguous substrings of s , and set $f(s) = |V(s)|$. Define

$$a(n) = \max_{s \in \{0, 1\}^n} f(s).$$

We use the convention $a(0) = 0$.

Lemma 1.2 (Leading-bit reduction). *For every $n \geq 2$, every optimal string for $a(n)$ begins with 1.*

Proof. Let $s = 0u$ have length n , and set $t = 1u$. Every substring of s not beginning at the first position is also a substring of t . A substring of s beginning at the first position has the form $0v$, where v is a prefix of u ; if v is nonempty, then $\text{val}(0v) = \text{val}(v)$ and v occurs in t beginning at the second position. Thus changing the first bit from 0 to 1 cannot lose any positive value.

If u contains a zero, the value 0 is still represented in t , and the full word t represents a positive integer whose standard representation has length n ; this value is not represented in s . Hence $f(t) > f(s)$.

It remains only to consider the case in which u contains no zero, i.e. $u = 1^{n-1}$. If $u = 1^{n-1}$, then $V(s) = \{0, 1, 3, \dots, 2^{n-1} - 1\}$ has size n . But the string $1^x 0^y$ with $x = \lfloor n/2 \rfloor$ and $y = \lceil n/2 \rceil$ has at least x all-one values, one zero value, and xy mixed values $\text{val}(1^a 0^b)$ distinguished by their odd parts and 2-adic valuations. Thus it has $xy + x + 1 > n$ distinct values for every $n \geq 2$. Hence such an s is not optimal either. $\square$

The first twenty values of the sequence are

1, 3, 5, 7, 10, 13, 17, 22, 27, 33, 40, 47, 55, 64, 73, 83, 94, 106, 118, 131.

Prior to the present work, values for $n \leq 80$ were certified by M. Fuller using an A* search and updated in the OEIS log in 2025. No asymptotic results were available in the literature.

Main theorem

The following theorem collects the main rigorous contributions of the paper. Heuristic observations not covered by this statement are separated later in Section 13.

Theorem 1.3 (Main results). *The sequence $a(n)$ has the following properties.*

(i) *The asymptotic limit exists and equals*

$$\lim_{n \rightarrow \infty} \frac{a(n)}{n^2} = \frac{1}{2}.$$

More precisely, for every $n \geq 36$,

$$\frac{n^2}{2} - \frac{13}{4}n^{3/2} \leq a(n) \leq \frac{n(n+1)}{2}.$$

- (ii) *There are strings s_n with $f(s_n) \geq n^2/2 - O(n^{3/2})$ and proportion of ones $1 - O(n^{-1/2})$. Conversely, every optimal string contains $O(n^{3/4})$ zeros.*
- (iii) *The sequence is convex: $a(n+1) - 2a(n) + a(n-1) \geq 0$ for every $n \geq 2$. Moreover $\Delta^2 a(n) = O(n^{3/4})$.*
- (iv) *The values of $a(n)$, and the complete set of optimal strings, are certified for $1 \leq n \leq 111$: Fuller's A* search covers $n \leq 80$, and the branch-and-bound computation in this work covers $81 \leq n \leq 111$.*
- (v) *If two consecutive zero-separators of a binary string have equal length s , and the three adjacent one-block lengths are b_i, b_{i+1}, b_{i+2} , then they force $\min(b_i, b_{i+1}) \min(b_{i+1}, b_{i+2})$ duplicate substring values of the form $\text{val}(1^j 0^s 1^k)$.*

Proof. Part (i) is Theorem 5.4, with the explicit bounds from Theorems 3.1 and 5.5. Part (ii) is Proposition 6.1 and Proposition 6.3. Part (iii) is Theorem 9.1 together with Corollary 9.7. Part (iv) is Theorem 12.4. Part (v) is Theorem 11.1. $\square$

Proof strategy and contributions

- (i) The upper bound is the elementary count of possible leading-1 substrings; the matching lower bound is an explicit family of strings whose internal one-block lengths form a strictly decreasing partition. This avoids probabilistic existence arguments and gives explicit constants.
- (ii) The structural bounds convert the deficit $n(n+1)/2 - a(n)$ into restrictions on zeros and run lengths. This gives density bounds for optimal strings and explains why long monochromatic runs are forced to be sublinear.
- (iii) Convexity is proved by comparing an optimal string with its prefix and suffix. A zero-dichotomy lemma isolates the only obstruction caused by the value 0.
- (iv) The exact computation is certified by combining Fuller's A* search for $n \leq 80$ with a branch-and-bound search for $81 \leq n \leq 111$. The branch-and-bound pruning rule is a theorem: the suffix can add at most a certified residual contribution plus a controlled number of crossing values. The suffix-automaton refinement only strengthens this upper bound.

- (v) The separator theorem is stated as a proved local obstruction. Stronger separator patterns observed in the data are recorded separately as observations and conjectures.

The accompanying data also record the related OEIS companion sequences obtained from optimal strings, including the smallest and largest optimal strings and the number of optima.

2 Definitions and notation

Throughout, binary strings are finite sequences over $\{0, 1\}$. The *binary value* of $w = w_1 \cdots w_k \in \{0, 1\}^k$ is

$$\text{val}(w) = \sum_{i=1}^k w_i 2^{k-i}.$$

A *substring* of $s = s_1 \cdots s_n$ is any $s_i s_{i+1} \cdots s_j$ with $1 \leq i \leq j \leq n$; we write $s[i..j]$ for this substring. The value set is $V(s) = \{\text{val}(s[i..j]) : 1 \leq i \leq j \leq n\}$.

The *run-length encoding* (RLE) of s is its decomposition into maximal constant blocks. When s begins with 1, we write $s = 1^{b_1} 0^{c_1} 1^{b_2} 0^{c_2} \cdots$, where the b_i are one-block lengths and the c_i are zero-block lengths. The zero-block lengths c_i , including a possible terminal zero-block, are called *separators*. Write $N_1(s)$ and $N_0(s)$ for the total number of ones and zeros in s .

If s has 1-runs of lengths $\ell_1, \dots, \ell_p$ and 0-runs of lengths $m_1, \dots, m_q$, set $L(s) = \max_t \ell_t$ and $K(s) = \max_t m_t$, with the convention that the maximum of an empty set is 0.

The *first difference* is $\Delta a(n) = a(n) - a(n-1)$ and, for $n \geq 2$, the *second difference* is $\Delta^2 a(n) = \Delta a(n+1) - \Delta a(n) = a(n+1) - 2a(n) + a(n-1)$.

3 Upper bounds

Theorem 3.1 (Trivial upper bound). *For every $n \geq 1$, $a(n) \leq n(n+1)/2$. In particular, $\limsup_{n \rightarrow \infty} a(n)/n^2 \leq \frac{1}{2}$.*

Proof. Every distinct positive integer in $V(s)$ has a unique standard binary representation beginning with 1; distinct positive values therefore correspond to distinct leading-1 substrings of s .

If s contains at least one zero, then $0 \in V(s)$, represented only by substrings beginning with 0. At least one of the $n(n+1)/2$ substrings begins with 0, so the number of leading-1 substrings is at most $n(n+1)/2 - 1$, giving $f(s) \leq n(n+1)/2$.

If $s = 1^n$, then $0 \notin V(s)$ and $f(s) = n \leq n(n+1)/2$. □

Remark 3.2. The bound is tight at $n = 2$: $V(10) = \{0, 1, 2\}$, so $a(2) = 3 = 2 \cdot 3/2$.

Theorem 3.3 (Length-stratified upper bound). *Let $K(n) = \max\{\ell \geq 1 : 2^{\ell-1} \leq n - \ell + 1\}$. Then*

$$a(n) \leq 2^{K(n)} + \binom{n - K(n) + 1}{2}.$$

Proof. Stratify the distinct positive values by the length ℓ of their standard binary representation. The number D_ℓ of distinct positive values representable by length- ℓ substrings of s satisfies $D_\ell \leq \min(2^{\ell-1}, n - \ell + 1)$: the first bound counts binary strings of length ℓ starting with 1; the second counts length- ℓ windows in s . Adding at most 1 for the value 0:

$$a(n) \leq 1 + \sum_{\ell=1}^n \min(2^{\ell-1}, n - \ell + 1).$$

By definition of $K = K(n)$, the minimum equals $2^{\ell-1}$ for $\ell \leq K$ and $n - \ell + 1$ for $\ell > K$. Hence

$$1 + \sum_{\ell=1}^K (2^{\ell-1}) + \sum_{\ell=K+1}^n (n - \ell + 1) = 2^K + \binom{n - K + 1}{2}. \quad \square$$

Remark 3.4. Since $K(n) = \log_2 n + O(\log \log n)$, the bound equals $n^2/2 - O(n \log n)$, which is strictly sharper than Theorem 3.1 for every $n \geq 3$.

4 The elementary lower bound

Theorem 4.1 (Elementary lower bound). *For all $n \geq 2$,*

$$a(n) \geq \lfloor n/2 \rfloor \lceil n/2 \rceil + \lfloor n/2 \rfloor + 1 \geq \frac{n^2}{4} + \frac{n}{2}.$$

In particular, $\liminf_{n \rightarrow \infty} a(n)/n^2 \geq \frac{1}{4}$.

Proof. Set $x = \lfloor n/2 \rfloor$, $y = \lceil n/2 \rceil$, and consider $s = 1^x 0^y$. Its substrings fall into three disjoint families:

- (a) **All-ones:** $s[i..j]$ with $j \leq x$, giving values $2^k - 1$ for $k = 1, \dots, x$: these are x distinct odd positive integers.
- (b) **All-zeros:** $s[i..j]$ with $i > x$, all having value 0: one distinct value.
- (c) **Mixed:** $s[i..j]$ with $i \leq x < j$, of the form $1^a 0^b$ with value $(2^a - 1) \cdot 2^b$, where $a \in \{1, \dots, x\}$ and $b \in \{1, \dots, y\}$. These xy values are pairwise distinct: for fixed b , different a give different odd parts $(2^a - 1)$; for different b , the 2-adic valuations differ. Moreover, all mixed values are even (disjoint from all-ones) and positive (disjoint from zero).

Thus $f(s) = x + 1 + xy = x(y + 1) + 1$. With $x = \lfloor n/2 \rfloor$ and $y = \lceil n/2 \rceil$: for even $n = 2m$, $f(s) = m(m + 1) + 1 = n^2/4 + n/2 + 1$; for odd $n = 2m + 1$, $f(s) = m(m + 2) + 1 = (n + 1)^2/4 > n^2/4 + n/2$. $\square$

5 The asymptotic limit

The elementary lower bound $a(n) \geq n^2/4 + O(n)$ and the upper bound $a(n) \leq n(n+1)/2$ leave the leading constant of $a(n)/n^2$ undetermined within $[1/4, 1/2]$. We now show it equals $1/2$ by constructing binary strings with $f(s) \geq n^2/2 - O(n^{3/2})$.

5.1 Strictly decreasing partitions

Lemma 5.1 (Decreasing partition). *Let $k \geq 2$ and $M \geq k(k+1)/2$ (equivalently, $M/k \geq (k+1)/2$). There exist strictly decreasing positive integers $b_1 > b_2 > \dots > b_k \geq 1$ with $\sum_{i=1}^k b_i = M$. Moreover, they may be chosen with $|b_i - M/k| \leq (k+1)/2$ for all i .*

Proof. Set $q = \lfloor M/k - (k-1)/2 \rfloor$ and $r = M - kq - k(k-1)/2$.

Step 1: $0 \leq r < k$. From the floor definition, $q \leq M/k - (k-1)/2 < q + 1$. Multiplying by k : $kq \leq M - k(k-1)/2 < k(q+1)$, so $0 \leq r < k$.

Step 2: Construction. Define

$$b_i = \begin{cases} q + k - i + 1, & 1 \leq i \leq r, \\ q + k - i, & r < i \leq k. \end{cases}$$

Step 3: Strict decrease. For consecutive $i, i+1$ both in the first case: $b_i - b_{i+1} = 1 > 0$. At the transition $i = r$ (if $r \geq 1$): $b_r - b_{r+1} = (q + k - r + 1) - (q + k - r - 1) = 2 > 0$. Both in the second case: $b_i - b_{i+1} = 1 > 0$.

Step 4: Sum equals M .

$$\begin{aligned} \sum_{i=1}^k b_i &= \sum_{i=1}^r (q + k - i + 1) + \sum_{i=r+1}^k (q + k - i) \\ &= \sum_{i=1}^k (q + k - i) + r = kq + \frac{k(k-1)}{2} + r = M. \end{aligned}$$

Step 5: Positivity. From $M \geq k(k+1)/2$, we get $M/k - (k-1)/2 \geq 1$, so $q \geq 1$ and $b_k = q \geq 1$.

Step 6: Proximity. All $b_i \in [q, q+k]$ and $M/k \in [q + (k-1)/2, q + (k+1)/2]$, so $|b_i - M/k| \leq (k+1)/2$. $\square$

5.2 The explicit construction

Fix $n \geq 64$ and set

$$k = \lfloor \sqrt{n}/4 \rfloor, \quad M = n - (k-1). \quad (1)$$

Since $n \geq 64$, we have $k \geq 2$. Moreover, $k-1 < \sqrt{n}/4 \leq n/2$, so $M > n/2$; and since $k(k+1) \leq 2k^2 \leq n/8 \leq n$, we have $M \geq n/2 \geq k(k+1)/2$, so Lemma 5.1 applies.

Choose $b_1 > b_2 > \dots > b_k \geq 1$ with $\sum b_i = M$ and $|b_i - M/k| \leq k$, as guaranteed by Lemma 5.1. Define the binary string

$$s = \underbrace{1^{b_1}}_{\text{block 1}} 0 \underbrace{1^{b_2}}_{\text{block 2}} 0 \dots 0 \underbrace{1^{b_k}}_{\text{block } k}, \quad (2)$$

consisting of k one-blocks separated by $k-1$ single zeros. Its length is $\sum b_i + (k-1) = M + (k-1) = n$.

5.3 An injective family of substrings

For integers i, j, α, β satisfying

$$1 \leq i, \quad j \geq i+2, \quad j \leq k, \quad 1 \leq \alpha \leq b_i, \quad 1 \leq \beta \leq b_j, \quad (3)$$

define $u_{i,j,\alpha,\beta}$ to be the substring of s beginning α positions before the end of block i and ending after the first β ones of block j . Explicitly, its run-length encoding is

$$u_{i,j,\alpha,\beta} = 1^\alpha 0 1^{b_{i+1}} 0 \dots 0 1^{b_{j-1}} 0 1^\beta. \quad (4)$$

The condition $j \geq i+2$ ensures at least one full interior block $b_{i+1}, \dots, b_{j-1}$. Every $u_{i,j,\alpha,\beta}$ starts and ends with 1, so it is the standard binary representation of a positive integer.

Lemma 5.2 (Injectivity). *The map $(i, j, \alpha, \beta) \mapsto u_{i,j,\alpha,\beta}$ is injective on the range (3).*

Proof. Suppose $u_{i,j,\alpha,\beta} = u_{i',j',\alpha',\beta'}$ as binary words. The run-length encoding of (4) is

$$(\alpha, 1, b_{i+1}, 1, \dots, b_{j-1}, 1, \beta).$$

Matching entries gives $\alpha = \alpha'$, $\beta = \beta'$, and $(b_{i+1}, \dots, b_{j-1}) = (b_{i'+1}, \dots, b_{j'-1})$. Since $b_1 > \dots > b_k$ is strictly decreasing, every consecutive subsequence occurs in exactly one position, so $i = i'$ and $j = j'$. $\square$

5.4 Quantitative lower bound

Theorem 5.3 (Quantitative lower bound). *For every $n \geq 64$, $a(n) \geq n^2/2 - 15n^{3/2}$.*

Proof. By Lemma 5.2, the family $\{u_{i,j,\alpha,\beta}\}$ consists of pairwise distinct positive integers (each starts and ends with 1), so

$$a(n) \geq f(s) \geq \sum_{\substack{1 \leq i < j \leq k \\ j \geq i+2}} b_i b_j. \quad (5)$$

Expanding:

$$\sum_{\substack{i < j \\ j \geq i+2}} b_i b_j = \frac{(\sum b_i)^2 - \sum b_i^2}{2} - \sum_{i=1}^{k-1} b_i b_{i+1}. \quad (6)$$

Step 1. Since $\sum b_i = M = n - k + 1$ and $k \leq \sqrt{n}/4$:

$$\frac{M^2}{2} \geq \frac{(n - \sqrt{n}/4)^2}{2} \geq \frac{n^2}{2} - \frac{n^{3/2}}{4}.$$

Step 2. Writing $b_i = M/k + d_i$ with $|d_i| \leq k$: $\sum b_i^2 = M^2/k + \sum d_i^2 \leq M^2/k + k^3$. Since $k \geq \sqrt{n}/8$ (as $\lfloor x \rfloor \geq x/2$ for $x \geq 2$), $M^2/k \leq n^2/(\sqrt{n}/8) = 8n^{3/2}$ and $k^3 \leq n^{3/2}/64$, giving

$$\sum_{i=1}^k b_i^2 < 9n^{3/2}. \quad (7)$$

Step 3. By the AM–GM inequality, $b_i b_{i+1} \leq (b_i^2 + b_{i+1}^2)/2$, so

$$\sum_{i=1}^{k-1} b_i b_{i+1} \leq \sum_{i=1}^k b_i^2 < 9n^{3/2}. \quad (8)$$

Conclusion. Substituting into (6):

$$a(n) \geq \frac{n^2}{2} - \frac{n^{3/2}}{4} - \frac{9n^{3/2}}{2} - 9n^{3/2} = \frac{n^2}{2} - \frac{55}{4}n^{3/2} > \frac{n^2}{2} - 15n^{3/2}. \quad \square$$

Theorem 5.4 (Limit). $\lim_{n \rightarrow \infty} \frac{a(n)}{n^2} = \frac{1}{2}$.

Proof. Theorem 3.1 gives $\limsup a(n)/n^2 \leq 1/2$. Theorem 5.3 gives $a(n)/n^2 \geq 1/2 - 15/\sqrt{n} \rightarrow 1/2$. The squeeze theorem gives the result. $\square$

5.5 A sharper quantitative bound

The constant 15 in Theorem 5.3 is an artifact of the conservative parameter choice $k = \lfloor \sqrt{n}/4 \rfloor$. By choosing $k = \lceil \sqrt{n} \rceil$ instead, we reduce the constant by a factor exceeding 4.

Theorem 5.5 (Sharper quantitative lower bound). *For every $n \geq 36$, $a(n) \geq n^2/2 - (13/4)n^{3/2}$.*

Proof. The construction (2) and Lemmas 5.1–5.2 are unchanged; only the parameter choice differs. Set $k = \lceil \sqrt{n} \rceil$ and $M = n - k + 1$. Then $\sqrt{n} \leq k \leq \sqrt{n} + 1$.

Lemma 5.1 applies for $n \geq 36$. We need $M \geq k(k+1)/2$. Since $k \leq \sqrt{n} + 1$, this requires $n - \sqrt{n} \geq \frac{1}{2}(\sqrt{n} + 1)(\sqrt{n} + 2) = n/2 + 3\sqrt{n}/2 + 1$, i.e., $n/2 \geq 5\sqrt{n}/2 + 1$, which holds for $\sqrt{n} \geq 6$, i.e., $n \geq 36$.

Variance bound. By Lemma 5.1, each b_i lies in $[q, q+k]$, giving

$$\sum_{i=1}^k b_i^2 \leq \frac{M^2}{k} + \frac{k(k+1)^2}{4}. \quad (9)$$

By AM–GM, $\sum_{i=1}^{k-1} b_i b_{i+1} \leq \sum b_i^2$. Substituting into (6):

$$a(n) \geq \frac{M^2}{2} - \frac{3M^2}{2k} - \frac{3k(k+1)^2}{8}.$$

Term-by-term estimates using $\sqrt{n} \leq k \leq \sqrt{n} + 1$ and $M = n - k + 1$:

- (i) $\frac{M^2}{2} \geq \frac{(n - \sqrt{n})^2}{2} = \frac{n^2}{2} - n^{3/2} + \frac{n}{2}.$
- (ii) $\frac{3M^2}{2k} \leq \frac{3n^2}{2\sqrt{n}} = \frac{3}{2} n^{3/2}.$
- (iii) Expanding $(\sqrt{n} + 1)(\sqrt{n} + 2)^2 = n^{3/2} + 5n + 8\sqrt{n} + 4$ gives

$$\frac{3k(k+1)^2}{8} \leq \frac{3}{8} n^{3/2} + \frac{15}{8} n + 3\sqrt{n} + \frac{3}{2}.$$

Combining (i)–(iii):

$$a(n) \geq \frac{n^2}{2} - \frac{23}{8} n^{3/2} - \frac{11}{8} n - 3\sqrt{n} - \frac{3}{2}. \quad (10)$$

Tail bound. We claim

$$\frac{11}{8} n + 3\sqrt{n} + \frac{3}{2} \leq \frac{3}{8} n^{3/2} \quad (n \geq 36). \quad (11)$$

Equivalently, $11n + 24\sqrt{n} + 12 \leq 3n^{3/2}$. At $n = 36$ the right side is 648 and the left is 552, so the inequality holds. The derivative of the difference, $\frac{9}{2}\sqrt{n} - 11 - 12/\sqrt{n}$, is positive for $n \geq 36$ (at $n = 36$: $27 - 11 - 2 = 14 > 0$) and increasing. Hence (11) holds for all $n \geq 36$.

Combining (10) and (11):

$$a(n) \geq \frac{n^2}{2} - \frac{23}{8} n^{3/2} - \frac{3}{8} n^{3/2} = \frac{n^2}{2} - \frac{13}{4} n^{3/2}. \quad \square$$

Corollary 5.6. *For all $n \geq 36$: $\frac{1}{2} - \frac{13}{4} n^{-1/2} \leq a(n)/n^2 \leq \frac{1}{2} + \frac{1}{2n}$.*

Remark 5.7. The constant $13/4$ is not expected to be optimal. Optimising the construction and its partition bookkeeping is a separate problem; the present paper uses only the explicit bound proved in Theorem 5.5.

Remark 5.8. Theorem 5.5 is non-trivially positive for $n \geq 43$, compared to $n \geq 901$ for Theorem 5.3; the improvement is mainly useful because it makes the asymptotic estimate visible already in the exact computational range.

6 Density of ones

6.1 Near-optimal strings

Proposition 6.1. *There exists a family of strings s_n with $f(s_n) \geq n^2/2 - O(n^{3/2})$ and $N_1(s_n)/n = 1 - O(n^{-1/2})$.*

Proof. For $n \geq 64$, take s_n from (2). By Theorem 5.3, $f(s_n) \geq n^2/2 - 15n^{3/2}$. The string contains exactly $k - 1 \leq \sqrt{n}/4$ zeros, so $N_1(s_n)/n \geq 1 - 1/(4\sqrt{n})$. The finitely many smaller values of n may be filled in arbitrarily. $\square$

6.2 The zero-position penalty

Lemma 6.2 (Zero-position bound). *Let $s \in \{0, 1\}^n$ with $z \geq 1$ zeros at positions $p_1 < \dots < p_z$. Then*

$$f(s) \leq \frac{n(n+1)}{2} - \sum_{r=1}^z (n - p_r + 1) + 1.$$

In particular, $f(s) \leq n(n+1)/2 - z(z+1)/2 + 1$.

Proof. Since $z \geq 1$, the value 0 is in $V(s)$ (any single zero bit is a substring representing 0). Every other element of $V(s)$ is positive; for each such x , choose a minimum-length representative w_x . By Lemma 9.2 (stated in Section 9), w_x starts with 1. This map is injective, so distinct positive values correspond to distinct leading-1 substrings.

Each zero at position p_r eliminates $n - p_r + 1$ potential leading-1 starts (all substrings beginning there start with 0). Hence the number of leading-1 substrings is at most $n(n+1)/2 - \sum_r (n - p_r + 1)$, and

$$f(s) = |\{0\}| + |\{x \in V(s) : x > 0\}| \leq 1 + \frac{n(n+1)}{2} - \sum_{r=1}^z (n - p_r + 1).$$

The sum is minimized by placing the zeros in the rightmost available positions, since each summand decreases as p_r increases. Thus the minimum is achieved by $p_r = n - z + r$, giving $\sum_r (z - r + 1) = z(z+1)/2$. $\square$

6.3 Optimal strings have few zeros

Proposition 6.3. *Every optimal string s_n^* satisfies $N_0(s_n^*) = O(n^{3/4})$, hence $N_1(s_n^*)/n = 1 - O(n^{-1/4})$.*

Proof. Let $z_n = N_0(s_n^*)$. By Theorem 4.1, $a(n) > n = f(1^n)$ for $n \geq 5$, so an optimal string has $z_n \geq 1$ in the range relevant to the asymptotic claim. For $n \geq 36$, Theorem 5.5 and Lemma 6.2 give

$$\frac{n^2}{2} - \frac{13}{4} n^{3/2} \leq a(n) \leq \frac{n(n+1)}{2} - \frac{z_n(z_n+1)}{2} + 1.$$

Hence $z_n^2 \leq z_n(z_n+1) \leq (13/2) n^{3/2} + n + 4$, giving $z_n = O(n^{3/4})$. $\square$

7 Upper bounds from run structure

Lemma 6.2 treats all zeros equally. We now refine the bound using the *run structure* of the zeros.

Lemma 7.1 (Run-sensitive bound). *Let $s \in \{0, 1\}^n$ with $z \geq 1$ zeros. Let the maximal zero-runs of s have lengths $c_1, \dots, c_m$, with the i -th run beginning at position a_i and $r_i = n - (a_i + c_i) + 1$ positions to its right. Then*

$$f(s) \leq \frac{n(n+1)}{2} - \sum_{i=1}^m \left(c_i r_i + \binom{c_i+1}{2} \right) + 1.$$

Proof. Apply Lemma 6.2 and group by zero-runs. The i -th run occupies positions $a_i, \dots, a_i + c_i - 1$, contributing $\sum_{t=0}^{c_i-1} (n - a_i - t + 1) = c_i(n - a_i + 1) - c_i(c_i - 1)/2 = c_i r_i + \binom{c_i+1}{2}$ to the penalty. $\square$

Corollary 7.2 (Fragmentation penalty). *If s has $z \geq 1$ zeros in m maximal runs, then*

$$f(s) \leq \frac{n(n+1)}{2} - \frac{z(z+1)}{2} - \binom{m}{2} + 1.$$

Proof. Since consecutive zero-runs are separated by at least one 1, for $i < m$: $r_i \geq \sum_{j>i} c_j + (m-i)$. Hence $\sum_i c_i r_i \geq \sum_{i<j} c_i c_j + \sum_{i=1}^{m-1} c_i(m-i)$. The identity $\sum_{i<j} c_i c_j + \sum_i \binom{c_i+1}{2} = \binom{z+1}{2}$ holds since $z = \sum c_i$. Finally, $\sum_{i=1}^{m-1} c_i(m-i) \geq \sum_{i=1}^{m-1} (m-i) = \binom{m}{2}$ since each $c_i \geq 1$. $\square$

Remark 7.3. Corollary 7.2 shows that fragmenting zeros into more runs always reduces the upper bound, regardless of positions. Each additional zero-run incurs a penalty of at least $m-1$.

8 The run-length frequency bound

Theorem 8.1 (Run-length frequency bound). *Every $s \in \{0,1\}^n$ satisfies*

$$f(s) \leq 1 + L(s) + \binom{n+1}{2} - \sum_{t=1}^p \binom{\ell_t+1}{2} - \sum_{t=1}^q \binom{m_t+1}{2}. \quad (12)$$

Proof. Let $V_1(s)$ be the set of values representable by an all-1 substring, and let $V_m(s)$ be the set of remaining positive values. Then $V(s)$ is contained in the disjoint union $\{0\} \cup V_1(s) \cup V_m(s)$; the singleton $\{0\}$ is harmless even when $0 \notin V(s)$.

Counting V_1 . An all-1 substring of length j has value $2^j - 1$, depending only on j . Hence $V_1(s) = \{2^j - 1 : 1 \leq j \leq L(s)\}$ and $|V_1(s)| = L(s)$.

Counting V_m . Every value in $V_m(s)$ is represented by a substring containing both 0 and 1. Hence $|V_m(s)|$ is at most the number of pairs (i, j) with $1 \leq i \leq j \leq n$ such that $s[i..j]$ contains both symbols. This equals $\binom{n+1}{2}$ minus the count of monochromatic substring pairs. A maximal 1-run of length ℓ_t contributes $\binom{\ell_t+1}{2}$ monochromatic pairs, and similarly for 0-runs. Hence $|V_m| \leq \binom{n+1}{2} - \sum_t \binom{\ell_t+1}{2} - \sum_t \binom{m_t+1}{2}$.

Combining via $f(s) \leq 1 + |V_1| + |V_m|$ gives (12). $\square$

Corollary 8.2 (Concentration of run lengths). *For any optimum s of $a(n)$ with $n \geq 36$:*

$$\sum_{t=1}^p \binom{\ell_t+1}{2} + \sum_{t=1}^q \binom{m_t+1}{2} \leq \frac{13}{4} n^{3/2} + \frac{n}{2} + L(s) + 1.$$

Proof. Rearrange Theorem 8.1 with $f(s) = a(n)$ and apply the lower bound $a(n) \geq n^2/2 - (13/4)n^{3/2}$ (Theorem 5.5), using $\binom{n+1}{2} = n^2/2 + n/2$. $\square$

Corollary 8.3 (Longest 1-run). *For every optimum s of $a(n)$ with $n \geq 36$: $L(s) = O(n^{3/4})$.*

Proof. The term $\binom{L+1}{2}$ appears on the left of Corollary 8.2; all other terms are non-negative. Hence $\binom{L+1}{2} \leq (13/4)n^{3/2} + n/2 + L + 1$, giving $L^2/2 \leq (13/4)n^{3/2} + L + n/2 + 1$ and thus $L \leq \sqrt{13/2} n^{3/4} + O(n^{3/8})$. $\square$

Corollary 8.4 (Longest 0-run). *For every optimum s of $a(n)$ with $n \geq 36$: $K(s) = O(n^{3/4})$.*

Proof. The term $\binom{K+1}{2}$ appears on the left of Corollary 8.2. By Corollary 8.3, $L(s) = O(n^{3/4})$, so

$$\binom{K+1}{2} \leq \frac{13}{4} n^{3/2} + \frac{n}{2} + O(n^{3/4}) + 1.$$

Hence $K^2 = O(n^{3/2})$, giving $K = O(n^{3/4})$. $\square$

9 Convexity of the sequence

Theorem 9.1 (Convexity). *For every $n \geq 2$, $a(n+1) - 2a(n) + a(n-1) \geq 0$. Equivalently, $\Delta a(n)$ is nondecreasing.*

The proof requires two preliminary lemmas.

Lemma 9.2 (Canonical witness). *For any $s \in \{0,1\}^n$ and any positive $x \in V(s)$, the standard binary representation of x (the unique string starting with 1 of length $\lfloor \log_2 x \rfloor + 1$) occurs as a substring of s . Moreover, every minimum-length substring of s representing x equals this standard representation.*

Proof. Let w_x be any minimum-length substring of s representing x . If w_x started with 0, removing the leading zero gives a shorter representative, contradicting minimality. So w_x starts with 1 and has length at least $\lfloor \log_2 x \rfloor + 1$. By minimality, equality holds. Since both w_x and the standard representation start with 1, have the same length, and represent x , they are identical. In particular, the standard representation occurs as a substring of s . $\square$

Lemma 9.3 (Zero dichotomy). *Let $s \in \{0,1\}^n$ with $n \geq 2$, and let $p = s_1 \cdots s_{n-1}$, $q = s_2 \cdots s_n$. Then $0 \notin V(s) \setminus V(p)$ or $0 \notin V(s) \setminus V(q)$ (or both).*

Proof. If $0 \in V(s) \setminus V(p)$, then s contains a zero but p does not, so the only zero of s is at position n . If $0 \in V(s) \setminus V(q)$, then s contains a zero but q does not, so the only zero of s is at position 1. Since $n \geq 2$, positions 1 and n are distinct, so both conditions cannot hold simultaneously. $\square$

Proof of Theorem 9.1. Fix $n \geq 2$ and let $s = s_1 \cdots s_n$ be optimal for $a(n)$. Let $p = s_1 \cdots s_{n-1}$ and $q = s_2 \cdots s_n$. Since $V(p) \subseteq V(s)$ and $V(q) \subseteq V(s)$:

$$|V(s) \setminus V(p)| \geq a(n) - a(n-1) = \Delta a(n), \quad |V(s) \setminus V(q)| \geq a(n) - a(n-1) = \Delta a(n). \quad (13)$$

By Lemma 9.3, at least one of $V(s) \setminus V(p)$ and $V(s) \setminus V(q)$ contains only positive values. We show $a(n+1) \geq a(n) + \Delta a(n)$ in each case.

Case 1: $0 \notin V(s) \setminus V(p)$. Every $x \in V(s) \setminus V(p)$ is positive. By Lemma 9.2, the standard binary representation w_x (of length $\ell_x = \lfloor \log_2 x \rfloor + 1$, starting with 1) occurs as a substring of s .

w_x must end at position n . If w_x ended at position $j \leq n-1$, it would lie entirely within p , giving $x \in V(p)$, a contradiction.

Form $t = s0$ of length $n+1$. For each $x \in V(s) \setminus V(p)$, the string $w_x 0$ is a substring of t .

$w_x 0$ is the standard representation of $2x$. Since w_x starts with 1 and $2^{\ell_x-1} \leq x < 2^{\ell_x}$, we have $2^{\ell_x} \leq 2x < 2^{\ell_x+1}$, so $w_x 0$ starts with 1 and has length $\lfloor \log_2(2x) \rfloor + 1 = \ell_x + 1$.

$2x \notin V(s)$. Suppose $2x \in V(s)$. By Lemma 9.2, s contains the standard representation w_{2x} as a substring, say occupying positions $i, \dots, i + \ell_x$. Then w_x occupies positions $i, \dots, i + \ell_x - 1$ with $i + \ell_x \leq n$, so w_x ends at position $i + \ell_x - 1 \leq n-1$, placing w_x inside p . This gives $x \in V(p)$, a contradiction.

Injectivity. The map $x \mapsto 2x$ is injective.

Conclusion. The values $\{2x : x \in V(s) \setminus V(p)\}$ are pairwise distinct, lie in $V(t) \setminus V(s)$, and number at least $\Delta a(n)$. Hence $f(t) \geq a(n) + \Delta a(n)$.

Case 2: $0 \notin V(s) \setminus V(q)$. Every $x \in V(s) \setminus V(q)$ is positive. By Lemma 9.2, the standard representation w_x occurs in s .

w_x must start at position 1. If w_x started at position $i \geq 2$, it would lie entirely within $q = s_2 \cdots s_n$, giving $x \in V(q)$, a contradiction.

Form $t = 1s$ of length $n+1$. For each $x \in V(s) \setminus V(q)$, the string $1w_x$ is a substring of t (occupying positions $1, \dots, \ell_x + 1$). Set $y = 2^{\ell_x} + x$.

$1 w_x$ is the standard representation of y . Since w_x starts with 1, the string $1 w_x$ starts with 1 and has length $\ell_x + 1$. Now $2^{\ell_x} \leq y < 2^{\ell_x+1}$, so the length is correct.

$y \notin V(s)$. If $y \in V(s)$, by Lemma 9.2 the string $1 w_x$ occurs in s , say starting at position i . Then w_x occupies positions $i+1, \dots, i+\ell_x$ with $i+1 \geq 2$, so w_x lies within $q = s_2 \cdots s_n$, giving $x \in V(q)$, a contradiction.

Injectivity. If $\ell_x = \ell_{x'}$ then $y \neq y'$ since $x \neq x'$. If $\ell_x < \ell_{x'}$ then $y < 2^{\ell_x+1} \leq 2^{\ell_{x'}} \leq y'$.

Conclusion. The values $\{2^{\ell_x} + x : x \in V(s) \setminus V(q)\}$ are pairwise distinct, lie in $V(t) \setminus V(s)$, and number at least $\Delta a(n)$. Hence $f(t) \geq a(n) + \Delta a(n)$.

In both cases, $a(n+1) \geq a(n) + \Delta a(n) = 2a(n) - a(n-1)$. $\square$

Remark 9.4. Theorem 9.1 establishes $\Delta^2 a(n) \geq 0$. Computational data further suggest $\Delta^2 a(n) \leq 2$ for all $n \geq 2$ (see Section 13).

We now derive a non-trivial upper bound on $\Delta^2 a(n)$ by combining convexity with the global asymptotic lower bound.

Lemma 9.5 (First-difference upper bound). *For all $n \geq 1$, $\Delta a(n) \leq n$.*

Proof. Let t be any string of length n and $p = t_1 \cdots t_{n-1}$. Every value in $V(t) \setminus V(p)$ is represented only by substrings ending at position n , of which there are at most n . Hence $f(t) \leq f(p) + n \leq a(n-1) + n$, so $a(n) \leq a(n-1) + n$. $\square$

Theorem 9.6 (Second-difference upper bound). *Define the deficit $E(n) = n(n+1)/2 - a(n)$. Then for all $n \geq 2$ and every integer $1 \leq h < n$,*

$$\Delta^2 a(n) \leq \frac{h+1}{2} + \frac{E(n)}{h}.$$

Optimising in h gives $\Delta^2 a(n) \leq \sqrt{2E(n)} + O(1)$.

Proof. By convexity (Theorem 9.1), Δa is nondecreasing, so for any $h \geq 1$:

$$a(n) - a(n-h) = \sum_{j=0}^{h-1} \Delta a(n-j) \leq h \Delta a(n).$$

Since $a(n-h) \leq (n-h)(n-h+1)/2$, we obtain

$$\begin{aligned} \Delta a(n) &\geq \frac{a(n) - a(n-h)}{h} \geq \frac{n(n+1)/2 - E(n) - (n-h)(n-h+1)/2}{h} \\ &= \frac{h(2n-h+1)/2 - E(n)}{h} = n - \frac{h-1}{2} - \frac{E(n)}{h}. \end{aligned}$$

Combining with Lemma 9.5:

$$\Delta^2 a(n) = \Delta a(n+1) - \Delta a(n) \leq (n+1) - \left(n - \frac{h-1}{2} - \frac{E(n)}{h}\right) = \frac{h+1}{2} + \frac{E(n)}{h}.$$

Setting $h = \lfloor \sqrt{2E(n)} \rfloor$ gives $\Delta^2 a(n) \leq \sqrt{2E(n)} + O(1)$. $\square$

Corollary 9.7. $\Delta^2 a(n) = O(n^{3/4})$. *More precisely, for $n \geq 36$,*

$$\Delta^2 a(n) \leq \sqrt{\frac{13}{2} n^{3/2} + n} + O(1).$$

Proof. By Theorem 5.5, $E(n) \leq \frac{13}{4} n^{3/2} + \frac{n}{2}$ for $n \geq 36$. Apply Theorem 9.6. $\square$

Remark 9.8. Any improvement in the global deficit bound $E(n) = n(n+1)/2 - a(n)$ immediately tightens the second-difference bound via Theorem 9.6. If the conjectured $E(n) = \Theta(n^{3/2})$ is optimal, the method yields $\Delta^2 a(n) = O(n^{3/4})$, far short of the empirical bound $\Delta^2 a(n) \leq 2$. Closing that gap appears to require additional structural information.

10 The ones-weighted crossing bound

The following bound, which partitions the score of a string into a prefix contribution and a residual, provides the theoretical foundation for the branch-and-bound exact solver described in Section 12.

Lemma 10.1 (Ones-weighted crossing bound). *Let $p \in \{0, 1\}^k$ be a prefix and $e \in \{0, 1\}^m$ any extension, with $n = k + m$ and $s = pe$. Let $V_0(p) = \{\text{val}(p[i..j]) : 1 \leq i \leq j \leq k\}$. Then*

$$f(s) \leq |V_0(p)| + a(m) + N_1(p) \cdot m,$$

where $N_1(p) = \#\{i : p_i = 1\}$.

Proof. Write $V_+(p, e)$ for the set of values represented by substrings not entirely within p . Then $f(s) \leq |V_0(p)| + |V_+(p, e)|$.

Partition $V_+(p, e)$ by where each defining substring begins:

(i) *Suffix-only substrings* (starting at position $> k$) contribute at most $|V(e)| \leq a(m)$ values.
(ii) *Zero-leading crossing substrings* (starting at $i \leq k$ with $p_i = 0$, ending at $j > k$): stripping leading zeros either reduces to a suffix-only value or to a one-leading crossing value. These contribute no new values beyond those in (i) and (iii).

(iii) *One-leading crossing substrings* (starting at $i \leq k$ with $p_i = 1$, extending $j \in \{1, \dots, m\}$ bits into e): for fixed i , set $A_i = \text{val}(p[i..k]) \geq 1$. The value is $v_{i,j} = A_i \cdot 2^j + \text{val}(e[1..j])$. For $j_1 < j_2$: $v_{i,j_2} \geq A_i \cdot 2^{j_2} \geq 2A_i \cdot 2^{j_1} > (A_i + 1) \cdot 2^{j_1} - 1 \geq v_{i,j_1}$ (using $A_i \geq 1$), so the m values are strictly increasing in j . There are $N_1(p)$ one-positions, giving at most $N_1(p) \cdot m$ crossing values.

Combining: $|V_+| \leq a(m) + N_1(p) \cdot m$. $\square$

Remark 10.2. In the branch-and-bound solver, this bound is evaluated at every node of the search tree: the prefix p is the partially constructed string, and $a(m)$ is looked up from previously proven values or from Theorem 3.3. The bound is typically 30–50% tighter than the naive residual $n(n+1)/2 - |V_0(p)|$.

10.1 Suffix-automaton refinement

The crossing bound of Lemma 10.1 counts $N_1(p) \cdot m$ crossing values as potentially new. Many of these are already present inside the fixed prefix and can be detected via the suffix automaton (SAM) of p .

Lemma 10.3 (SAM savings). *Let $p \in \{0, 1\}^k$, let $e \in \{0, 1\}^m$, put $s = pe$, and let $\mathcal{S}$ be the suffix automaton of p . For each position $i \leq k$ with $p_i = 1$, define d_i to be the largest integer d such that the SAM state reached by the suffix $p[i..k]$ has a complete binary transition subtree of depth d . Equivalently, for every $1 \leq j \leq d$ and every binary string $w \in \{0, 1\}^j$, the word $p[i..k] \cdot w$ occurs in p , so its value already lies in $V_0(p)$. Then*

$$f(s) \leq |V_0(p)| + a(m) + \sum_{\substack{i=1 \\ p_i=1}}^k (m - d_i)^+,$$

where $(x)^+ = \max(x, 0)$.

Proof. By the proof of Lemma 10.1, each one-position i contributes at most m crossing values $v_{i,1}, \dots, v_{i,m}$. The suffix automaton represents exactly the substrings of the fixed prefix. Therefore, if the state reached by the suffix $p[i..k]$ has all binary paths of length j , then for every $w \in \{0, 1\}^j$ the word $p[i..k]w$ occurs as a substring of the fixed prefix and its value already lies in $V_0(p)$. For $j = 1, \dots, d_i$, these are precisely the first d_i possible crossing extensions from position i , so they are guaranteed duplicates. Subtracting them from the budget of m possible new crossing values gives at most $(m - d_i)^+$ new values from that position. Summing over all one-positions yields the result. $\square$

Remark 10.4. Computing all d_i via a bounded breadth-first search of depth $J \leq 3$ in the SAM takes $O(k \cdot 2^J)$ time per node of the search tree. In practice, this refinement reduces the crossing budget by 20–40% at moderate prefix lengths, and is the primary reason the branch-and-bound solver terminates within practical time for n up to 111.

11 Separator structure

The certified optimal strings for $81 \leq n \leq 111$ all have separator sequence beginning $(1, 2, 1, 1, \dots)$. We prove below that any two consecutive separators of equal length produce forced collisions (Theorem 11.1). This gives a rigorous obstruction to repeated adjacent separator lengths, but it is not by itself a proof of the full separator pattern.

11.1 Forced collisions from equal consecutive separators

The following theorem shows that two consecutive separators of the same length force a quadratic number of collisions.

Theorem 11.1 (Forced collisions from equal consecutive separators). *Let w be a binary string with ones-block lengths $b_1, b_2, b_3, \dots$ and separator lengths $s_1, s_2, \dots$, so that w begins $1^{b_1} 0^{s_1} 1^{b_2} 0^{s_2} 1^{b_3} \dots$. Suppose two consecutive separators have the same length: $s_i = s_{i+1} = s$ for some i . Set $J = \min(b_i, b_{i+1})$ and $K = \min(b_{i+1}, b_{i+2})$. Then the $J \cdot K$ values*

$$\{v_{j,k} = \text{val}(1^j 0^s 1^k) : 1 \leq j \leq J, 1 \leq k \leq K\}$$

are pairwise distinct and each appears at least twice as a substring of w . Explicitly, $v_{j,k} = (2^j - 1) \cdot 2^{s+k} + 2^k - 1 = 2^{j+s+k} - 2^{s+k} + 2^k - 1$.

Proof. Two occurrences. For $j \leq J$ and $k \leq K$, the pattern $P_{j,k} = 1^j 0^s 1^k$ (representing $v_{j,k}$) appears at two positions in w :

1. Crossing separator i : the last j ones of block i ($j \leq b_i$), then $0^s = 0^{s_i}$, then the first k ones of block $i+1$ ($k \leq b_{i+1}$).
2. Crossing separator $i+1$: the last j ones of block $i+1$ ($j \leq b_{i+1}$), then $0^s = 0^{s_{i+1}}$, then the first k ones of block $i+2$ ($k \leq b_{i+2}$).

These positions differ since $b_{i+1} \geq 1$.

Distinctness. Each $v_{j,k}$ has standard binary representation $1^j 0^s 1^k$ (a string of length $j + s + k$ starting with 1). If $v_{j_1, k_1} = v_{j_2, k_2}$, their standard representations are identical, so $j_1 + k_1 = j_2 + k_2$ (same total length) and the 0^s block occupies the same positions: $j_1 = j_2$, hence $k_1 = k_2$.

These are genuine collisions. Each $v_{j,k}$ appears at two distinct starting positions, so its substring occurrence is counted twice but contributes only once to $f(w)$. This represents $J \cdot K$ forced collisions. $\square$

Corollary 11.2. *If a string w with $z \geq 1$ zeros has $s_i = s_{i+1}$ for some i , then $f(w) \leq n(n+1)/2 - Z(w) + 1 - C(w) - J \cdot K$, where $p_1 < \dots < p_z$ are the zero positions, $Z(w) = \sum_{r=1}^z (n - p_r + 1)$ is the zero-position penalty (Lemma 6.2), $C(w) = \sum_j \binom{b_j+1}{2} - \max_j b_j$ counts one-block self-collisions, and J, K are as in Theorem 11.1.*

Proof. Let $N = n(n+1)/2$. By Lemma 9.2, every positive value has a standard representative beginning with 1. There are at most $N - Z(w)$ leading-1 substrings, and the value 0 contributes at most one further value.

Among the leading-1 substrings, the all-1 substrings contribute $\sum_j \binom{b_j+1}{2}$ occurrences but at most $\max_j b_j$ distinct values, so they account for at least $C(w)$ duplicate occurrences.

The $J \cdot K$ collisions of Theorem 11.1 are disjoint from these all-1 collisions because their standard representatives $P_{j,k} = 1^j 0^s 1^k$ contain a zero. They are also disjoint from zero-starting

substrings because each $P_{j,k}$ begins with 1. Therefore the leading-one substring count overcounts the positive values by at least $C(w) + JK$, and

$$f(w) \leq 1 + (N - Z(w)) - C(w) - JK. \quad \square$$

Remark 11.3. The theorem requires only that $s_i = s_{i+1}$; it applies to consecutive separators of any common length. Making two consecutive separators differ eliminates the collisions: the two occurrences of $P_{j,k}$ then cross zero-blocks of different lengths, producing the distinct values $\text{val}(1^j 0^{s_i} 1^k) \neq \text{val}(1^j 0^{s_{i+1}} 1^k)$.

For $n \approx 100$ with hypothetical separator sequence $(1, 1, \dots)$, the first pair $(s_1, s_2) = (1, 1)$ gives $J \cdot K = \min(b_1, b_2) \cdot \min(b_2, b_3) \approx 17 \cdot 14 = 238$ forced collisions, roughly 20% of the total deficit. Changing s_2 from 1 to 2 eliminates these collisions at the cost of one additional bit; separators ≥ 3 also work but waste an extra position. In the computed optima, later separator pairs have adjacent blocks small enough that the collision count $\min(b_i, b_{i+1}) \cdot \min(b_{i+1}, b_{i+2})$ is negligible, making the cost of an extra separator bit outweigh the observed savings. This is consistent with the pattern $(1, 2, 1, 1, 1, \dots)$ favoured by the data, but the infinite statement remains Conjecture 13.4.

12 Computational results

12.1 Scoring

For small n ($n \lesssim 30$), $f(s)$ is computed by inserting all substring values into a hash set. For larger n , we use the suffix array SA and LCP array (via Kasai's algorithm [2]):

$$f(s) = \sum_{\substack{0 \leq i < n \\ s[\text{SA}[i]] = 1}} (n - \text{SA}[i] - \text{LCP}[i]) + \mathbf{1}[0 \in s].$$

Here the suffix array is indexed from 0 and $\text{LCP}[0] = 0$. This formula counts, for each suffix in sorted order, the number of new leading-1 substrings it contributes beyond those shared with the preceding suffix (as measured by the LCP), then adds 1 if the string contains any zero (for the value 0). The standard suffix-array argument applies because every positive value is identified with its unique leading-1 representation: prefixes of a suffix that are not longer than its LCP with the preceding suffix have already occurred, and longer prefixes are new. Thus one scoring pass for a fixed string takes $O(n \log n)$ time, or $O(n)$ when the suffix array is constructed with the DivSufSort library [3]. This is distinct from the size of the branch-and-bound search tree considered below.

12.2 Branch-and-bound exact solver

To prove exact values of $a(n)$ beyond $n = 80$, we developed a branch-and-bound solver that constructs n -bit strings bit-by-bit, pruning branches whose upper bound falls below the current best known score.

Definition 12.1 (Upper bound at a search node). At a node where the prefix $p \in \{0, 1\}^k$ has been fixed (with $m = n - k$ remaining bits), let $B(m)$ be any certified upper bound for $a(m)$. The node upper bound is

$$U(p) = |V_0(p)| + B(m) + \sum_{\substack{i=1 \\ p_i=1}}^k (m - d_i)^+,$$

where $V_0(p)$ is the set of distinct values of substrings entirely within p . In a self-contained certificate one may take $B(m)$ from Theorem 3.3; tighter runs may replace this by exact values

only when those values are independently certified. The integer d_i is the SAM savings depth at position i (Lemma 10.3).

Since $B(m) \geq a(m)$, Lemmas 10.1 and 10.3 imply that $U(p)$ is a valid upper bound on $f(s)$ for any completion s of p .

The algorithm (pseudocode below) proceeds as follows:

1. Initialise the incumbent $L \leftarrow$ a known lower bound (from a heuristic search or a previously computed value for a nearby n).
2. Perform depth-first search over all possible bit assignments, maintaining a suffix automaton (SAM) for the current prefix to track $|V_0(p)|$ and the SAM savings d_i incrementally.
3. At each node, compute $U(p)$. In value-only mode, prune when $U(p) \leq L$; in `collect_all` mode, prune only when $U(p) < L$, so branches that may contain further optima are kept.
4. At a leaf ($k = n$), evaluate $f(s)$ exactly. If $f(s) > L$, update L and, in `collect_all` mode, clear the current optimum list; if $f(s) = L$, record s in that list.
5. On termination, $L = a(n)$.

```

BRANCHANDBOUND( $n$ )


---


if  $n = 1$ :    return 1
 $L \leftarrow$  initial lower bound
SEARCH(1, SAM1, 1, 1)
return  $L$ 

SEARCH( $p$ ,  $\mathcal{S}$ ,  $|V_0|$ ,  $N_1(p)$ )
   $k \leftarrow |p|$ ;    $m \leftarrow n - k$ 
  if  $k = n$ :    update  $L$  and optimum records;    return
  Compute  $d_i$  for each one-position  $i$  via SAM  $\mathcal{S}$ 
   $U \leftarrow |V_0| + B(m) + \sum_{i:p_i=1} (m - d_i)^+$ 
  if PRUNABLE( $U, L$ ):    return
  for  $b \in \{1, 0\}$ :
    Extend SAM:  $\mathcal{S}' \leftarrow$  SAMEXTEND( $\mathcal{S}, b$ )
     $|V'_0| \leftarrow$  updated distinct substring count
    SEARCH( $p \cdot b$ ,  $\mathcal{S}'$ ,  $|V'_0|$ ,  $N_1(p) + b$ )

```

Here PRUNABLE(U, L) means $U < L$ in `collect_all` mode and $U \leq L$ in value-only mode.

Remark 12.2 (Correctness). The solver’s correctness rests on two properties: (1) the upper bound $U(p) \geq f(s)$ for every completion s of prefix p (Lemmas 10.1 and 10.3), so pruning never discards a global optimum; and (2) the search exhaustively explores all branches not pruned. The SAM provides exact tracking of $|V_0(p)|$ (each SAM state corresponds to an equivalence class of right extensions; the total count of distinct substrings is maintained incrementally as each bit is appended).

The value search is run with initial prefix 1, which is valid by Lemma 1.2. For results that assert properties of all optimal strings, the solver is run in `collect_all` mode: branches with $U(p) = L$ are not pruned merely because one incumbent optimum is already known. Thus every completion with score $a(n)$ is either reached and recorded or lies in a subtree whose certified upper bound is below $a(n)$.

Remark 12.3 (Implementation and reproducibility). The implementation uses Python 3.13 with Numba [5] for JIT compilation of the inner scoring loop and the SAM operations. Parallel search is performed across multiple workers, each exploring a subtree rooted at a distinct prefix of length ≈ 20 . Workers share the incumbent L via atomic updates. This affects only the order in which leaves are visited; in `collect_all` mode the certificate is the final unordered set of strings attaining the maximum, not the discovery order. Typical running times range from seconds ($n \leq 90$) to ~ 48 hours ($n = 111$) on a 16-core workstation.

Source code is available at [6]. To reproduce the new extension, run the branch-and-bound solver for each target n with `collect_all=True`; the solver outputs the exact value, the complete list of optimal strings, and the number of nodes explored and pruned. The archived outputs for the extension are included in the repository under `results/branch_bound_exact/`, with one JSON certificate file `n_NNNN_results.json` per value of n ; these files record the optimal-string list, `num_optimal`, and the run metadata. The consolidated source-data file `data/cached_results.json` records the exact entries through $n = 111$, including Fuller’s $n \leq 80$ data and the branch-and-bound extension. The companion sequences A112510, A112511, and A156025 are then obtained respectively as the minimum, maximum, and cardinality of this list; A156022–A156024 follow from the corresponding value counts.

12.3 Exact values

Theorem 12.4 (Certified exact computation). *The values of $a(n)$ are certified for $1 \leq n \leq 111$. For each such n , the complete list of optimal strings is certified, and hence so are the corresponding values of A112510, A112511, and A156025. The values of A156022–A156024 then follow from the definitions.*

Proof. Fuller’s A* search certifies the range $1 \leq n \leq 80$, with the corresponding optimal-string and companion-sequence data recorded in the source table. The branch-and-bound extension is then certified sequentially for $n = 81, 82, \dots, 111$. At the run for a given n , every residual length $m < n$ has already been certified, so the computation described in Section 12.2, run in `collect_all` mode, may use the certified values for smaller residuals as exact upper bounds $B(m) = a(m)$. Hence the upper bound $U(p)$ is certified at every search node. Exhaustive exploration of all branches not pruned then proves the optimum. Since `collect_all` mode does not prune branches with $U(p) = a(n)$, every string attaining the optimum is recorded in the corresponding archived certificate in `results/branch_bound_exact/`. Taking the minimum, maximum, and cardinality of this list gives A112510, A112511, and A156025, respectively. $\square$

The full machine-readable table is included with the source data; the main text records only representative values and the growth ratio.

n	$a(n)$	$a(n)/n^2$	source
50	899	0.3596	Fuller/OEIS
80	2,423	0.3786	Fuller/OEIS
100	3,875	0.3875	certified here
111	4,826	0.3916	certified here

12.4 Exploratory lower bounds for $112 \leq n \leq 200$

For $112 \leq n \leq 200$, lower bounds on $a(n)$ were computed using two independent Metropolis–Hastings (MH) search methods:

1. *Unconstrained MH* — random bit-flip Markov chains on the full space of n -bit strings, accepting moves that do not decrease $f(s)$ (10^5 restarts, 10^5 iterations each).
2. *Constrained MH* — chains restricted to a fixed run-length template derived from empirical block-range constraints, with 2×10^5 steps per chain across 8 parallel chains.

Both methods agreed on the best value for every n in this range. These results are certified lower bounds (since $f(s)$ is evaluated exactly), but are not proved to be exact values of $a(n)$; the agreement across independent methods is recorded as evidence only and is not used in any proof.

12.5 Large-scale certified lower bounds

For $n \geq 3,000$, a suffix-array-guided local search iteratively improves a single incumbent string by identifying and resolving the highest-LCP collisions via targeted single-bit swaps. The table reports exact scores of explicit strings. Since each score is computed exactly, every listed value is a rigorous lower bound on $a(n)$, not a certified exact value of $a(n)$.

n	Lower bound on $a(n)$	$a(n)/n^2 \geq$
3,000	4,261,266	0.4735
10,000	48,490,001	0.4849
10^6	498,396,662,200	0.4984
10^9	499,943,911,277,037,650	0.49994
2×10^9	1,999,841,243,789,594,574	0.49996

The computation at $n = 2 \times 10^9$ scores a two-billion-bit string (≈ 238 MB as a NumPy array) using the DivSufSort suffix-array library [3].

Remark 12.5. Theorem 5.5 gives $a(n)/n^2 \geq 1/2 - 13/(4\sqrt{n})$, which reaches 0.4875 only at $n = 14,400$. At moderate values of n , the computational lower bounds lie well above this conservative theoretical estimate, indicating that the constant $13/4$ is not close to the behaviour of the explicit strings found in practice.

13 Empirical observations and open problems

13.1 Second differences

Observation 13.1. In the certified data through $n \leq 111$: $\Delta^2 a(n) \in \{0, 1, 2\}$. The value 2 occurs at $n = 52$ and $n = 71$. For heuristic values up to $n = 200$, $\Delta^2 a(n) \in \{0, 1, 2\}$ continues to hold, with additional occurrences of 2 at $n \in \{117, 157, 178, 191, 194\}$.

Theorem 9.1 establishes $\Delta^2 a(n) \geq 0$ for $n \geq 2$.

Conjecture 13.2. $\Delta^2 a(n) \leq 2$ for all $n \geq 2$.

13.2 The separator pattern

Observation 13.3. The certified data show that, for every $8 \leq n \leq 111$, every optimal n -bit string has first separator length 1. The threshold is sharp: exhaustive enumeration for $n \leq 7$ shows that, for $n = 4, 5, 6, 7$, some optimal strings have first separator length greater than 1.

For all $81 \leq n \leq 111$ (the extension beyond Fuller’s computation), every optimal n -bit string has separator sequence beginning $(1, 2, 1, 1, 1, 1, 1, 1)$. The same pattern holds for all best heuristic strings found through $n = 200$.

Theorem 11.1 gives a rigorous obstruction to equal adjacent separators: equal consecutive separators force $\min(b_i, b_{i+1}) \cdot \min(b_{i+1}, b_{i+2})$ collisions. For the first pair in the computed optima, these collisions are large enough to justify lengthening s_2 to 2; for later pairs, the adjacent blocks are too small to warrant the observed cost.

Conjecture 13.4. For every $n > 7$, every optimal n -bit string has first separator length 1. Moreover, for all sufficiently large n , every optimal n -bit string has separator sequence beginning $(1, 2, 1, 1, 1, 1, 1, 1, \dots)$.

13.3 Open problems

- (i) **Error term.** The best proven bound gives $a(n)/n^2 = 1/2 + O(n^{-1/2})$. Can the error term be improved to $O(n^{-\alpha})$ for some $\alpha > 1/2$? Computational data suggest $a(n) = n^2/2 - \Theta(n^{3/2})$.
- (ii) **Separator structure.** Prove or disprove Conjecture 13.4.
- (iii) **Second differences.** Prove $\Delta^2 a(n) \leq 2$ (Conjecture 13.2).
- (iv) **Leading block.** The empirical leading one-block length satisfies $b_1 \approx 0.19n$. The best proven upper bound is $b_1 = O(n^{3/4})$ (Corollary 8.3). Close this gap.
- (v) **Density.** The empirical ones-density is ≈ 0.85 for moderate n . The best proven lower bound is $1 - O(n^{-1/4})$ (Proposition 6.3). Prove a bound approaching $1 - O(n^{-1/2})$.
- (vi) **Exact computation.** Extend the certified exact range beyond $n = 111$.
- (vii) **Alphabet generalisation.** For an alphabet $\{0, \dots, q-1\}$, define $a_q(n)$ analogously. We have $a_q(n)/n^2 \rightarrow 1/2$ for every $q \geq 2$: the upper bound $a_q(n) \leq n(n+1)/2$ is alphabet-independent, and restricting to the sub-alphabet $\{0, q-1\}$ reduces to the binary case, giving $a_q(n) \geq a_2(n) \geq n^2/2 - O(n^{3/2})$. The deficit $n(n+1)/2 - a_q(n)$ appears to be much smaller for $q \geq 3$ (roughly $O(n \log n)$ versus $\Theta(n^{3/2})$ for $q = 2$), since short-substring collisions vanish once the alphabet exceeds 2. Determining the precise deficit order for $q \geq 3$ remains open.

Acknowledgements

The author thanks Martin Fuller for the original A* certification of $a(n)$ for $n \leq 80$ [1]. All computations were performed with Python 3.13 using the `numpy` [4], `numba` [5], and `pydivsufsort` [3] libraries.

References

- [1] OEIS Foundation Inc., *The On-Line Encyclopedia of Integer Sequences*, Sequence A112509, <https://oeis.org/A112509>, 2026. Values for $n \leq 80$ certified by an A* search of M. Fuller and updated in the OEIS log in 2025.
- [2] T. Kasai, G. Lee, H. Arimura, S. Arikawa, and K. Park, *Linear-time longest-common-prefix computation in suffix arrays and its applications*, in Proceedings of CPM 2001, LNCS 2089, Springer, 2001, pp. 181–192.
- [3] L. Abraham, `pydivsufsort`: Python bindings for DivSufSort, <https://github.com/louisabraham/pydivsufsort>, 2023.
- [4] C. R. Harris et al., *Array programming with NumPy*, Nature **585** (2020), 357–362.
- [5] S. K. Lam, A. Pitrou, and S. Seibert, *Numba: A LLVM-based Python JIT compiler*, Proceedings of LLVM-HPC 2015, ACM, 2015.
- [6] B. Clare, *A112509: computational exploration*, <https://github.com/axelsmagichammer/A112509>, 2026.